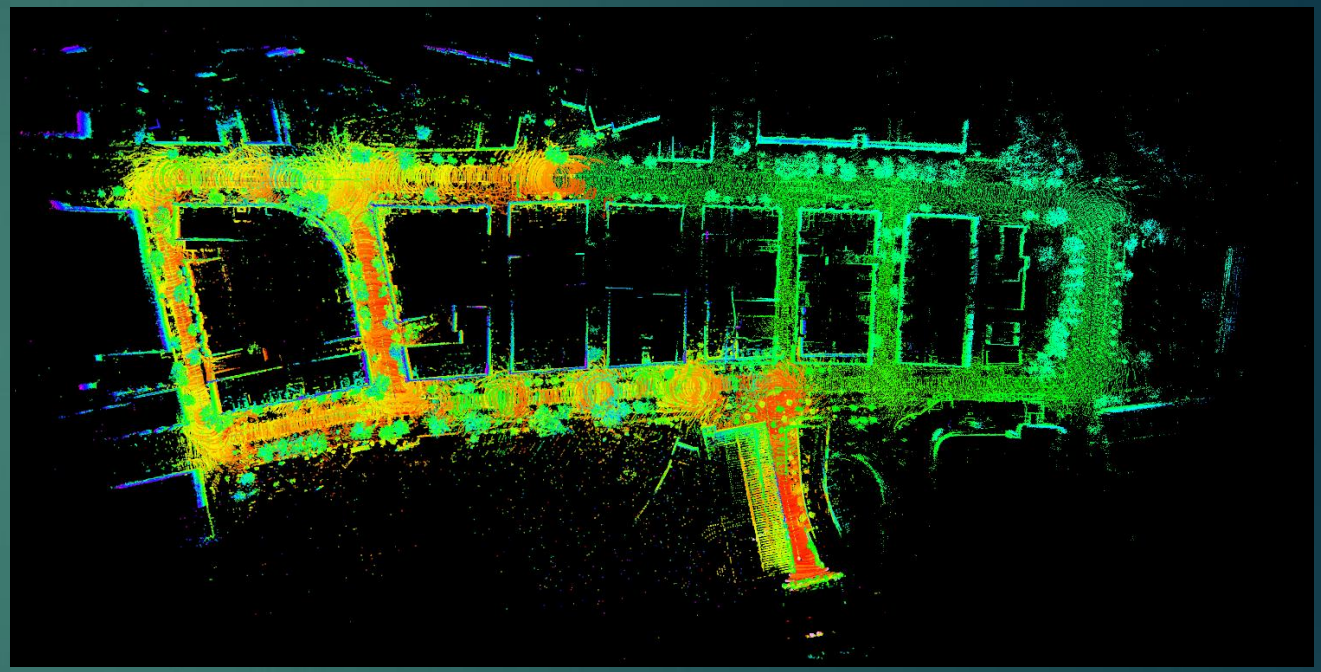
Evaluation of LeGO-LOAM: Lightweight and Ground-Optimized Lidar Odometry and Mapping



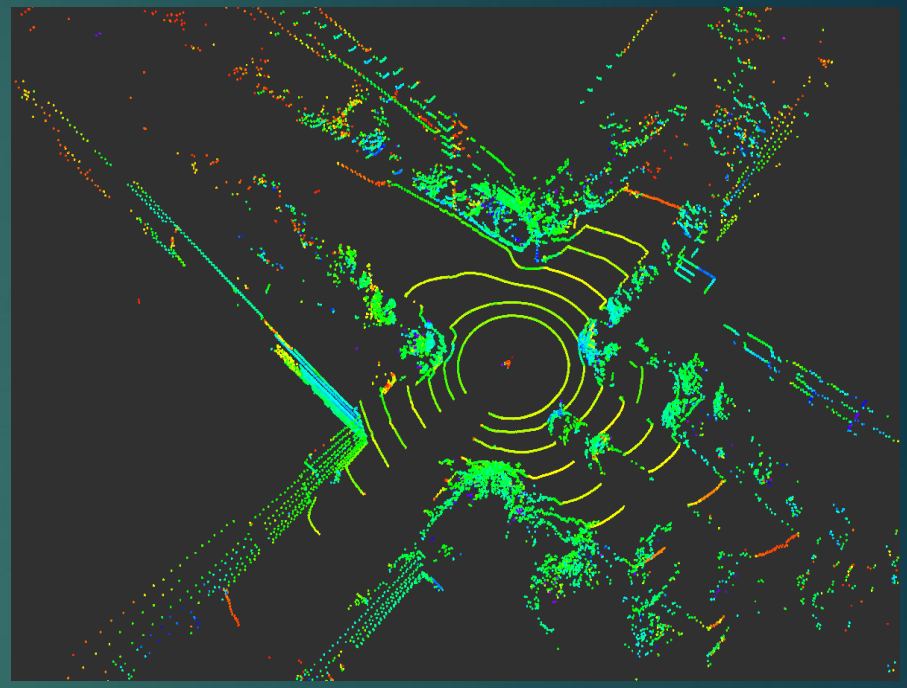


Team 5

Akshdeep Gill, Mahdi Edraki, Rohit Rajput, Tejaswini Bharatha

Introduction

- Lidar slam has an advantage over visual SLAM due to high precision in simple and static conditions
- Works on a very light system compared to V-SLAM
- The Lego-loam uses 3d lidar data to support real-time state estimation and mapping
- Performs point feature to edge/plane scan-matching to find correspondences between scans
- Feature extraction by calculating the roughness of a point in its local region
- Point cloud segmentation to remove unnecessary features points
- Lego-Loam does not assume the ground to be flat



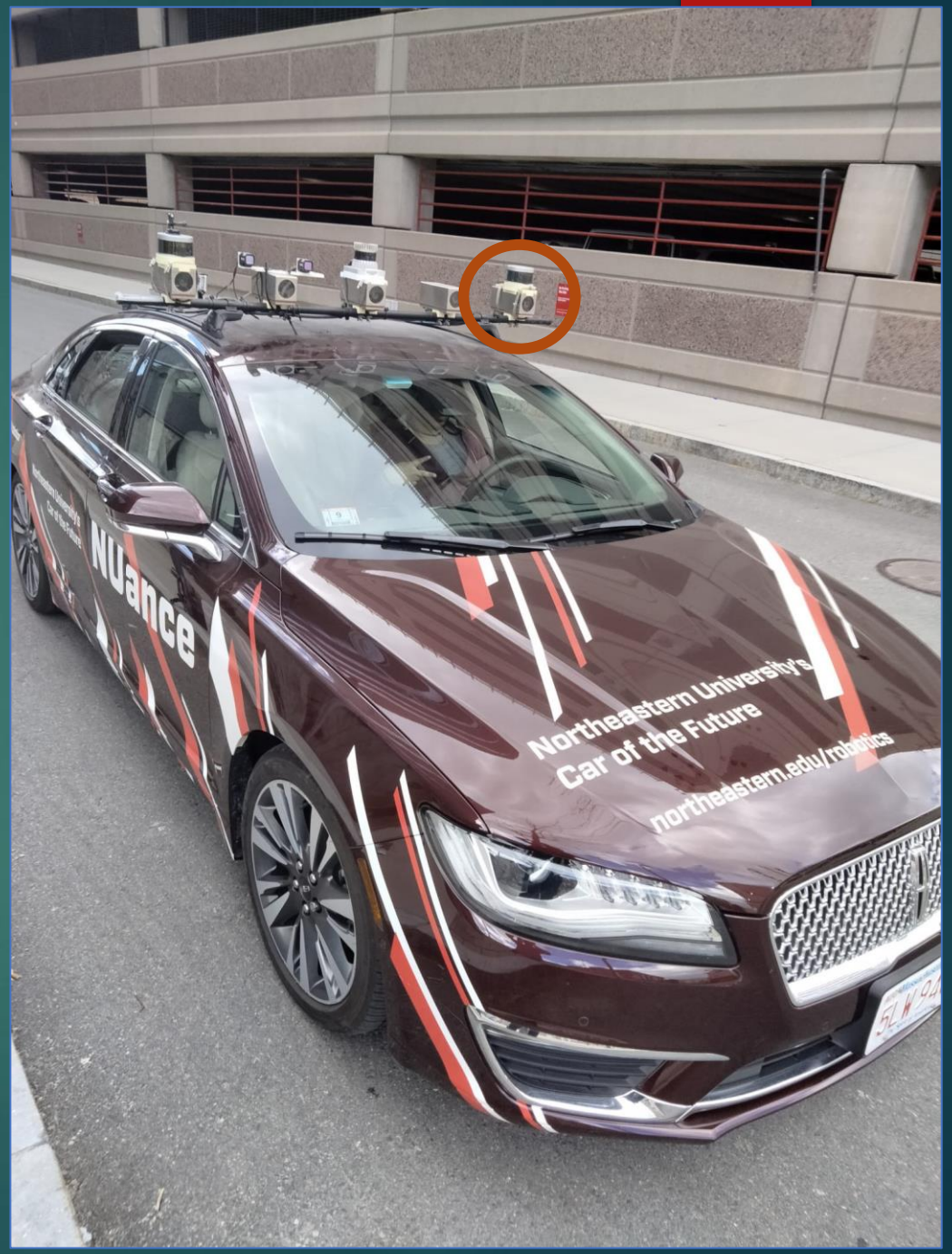
Data Collection

NUance car :

- Left velodyne VLP-16 lidar
  - range 100 m with accuracy ± 3cm
  - a 360° horizontal field of view a 30°
  - vertical field of view
  - +/- 15° up and down
- Vectornav IMU 9 axis
- GPS

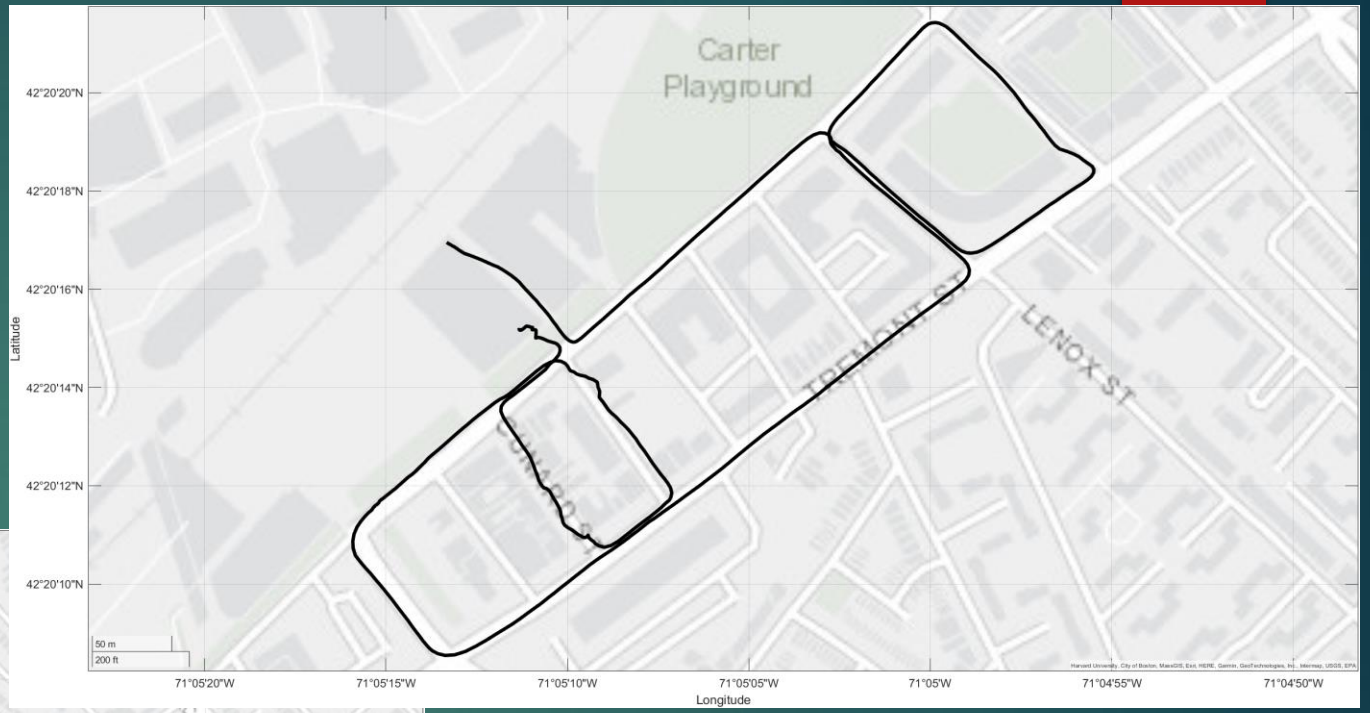


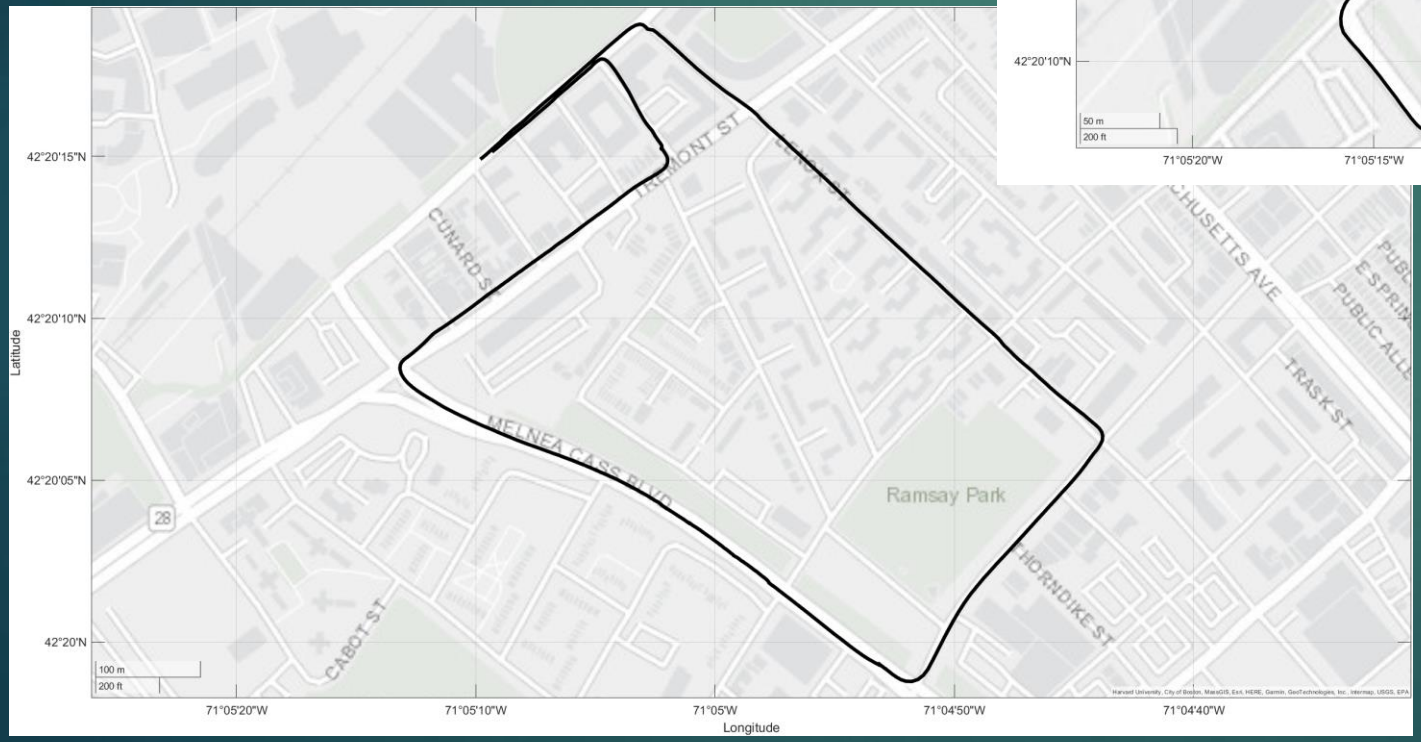




Datasets Path

- Dataset 1
  - Three loop closures





- Dataset 2
  - One long distance loop

Software System & Code

- System used:

  - Ubuntu 20.04
  - ROS noetic
  - Opencv4
  - Gtsam 4.0

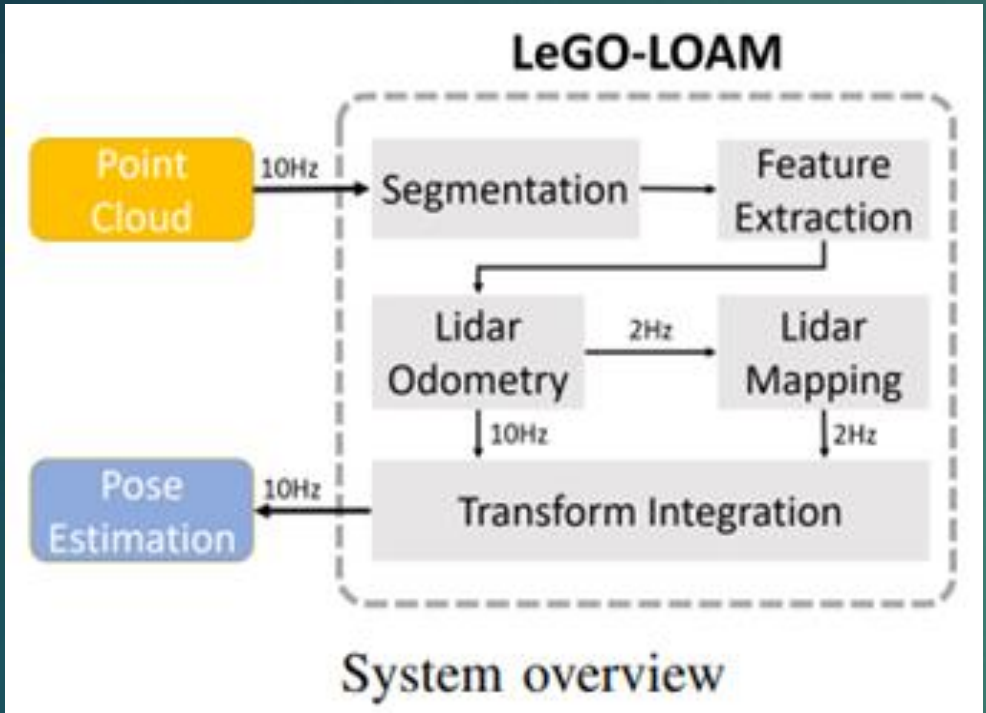




- Issues faced while build and run of lego-Loam

Issues faced	Resolving action
#include <opencv/cv.h> not compatable with current OpenCV	Used latest opencv core file include <opencv2/opencv.hpp>
Eigen:Index not supported in PCL	Updated gtsam to latest version
C++ 14 build error	Forcing cmake to use c++14 set(CMAKE_CXX_STANDARD_REQUIRED ON)
IMU Topic Changed	New IMU topic inserted in utility.h
RVIZ Mapping Error	Removed all '/' from sensor topics

Methodology

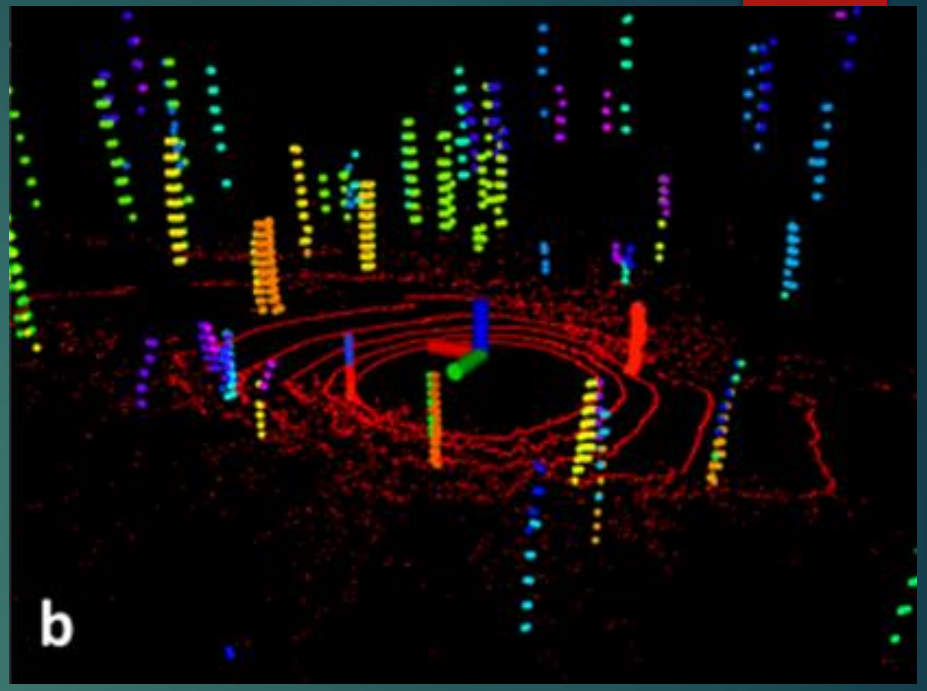


System overview

- Input from 3d Lidar outputs as 6 DOF pose estimation
- The proposed system seeks improved efficiency and accuracy for ground vehicles while reducing computation time.
- The LeGO-LOAM algorithm is divided into five modules:
  - Segmentation
  - Feature extraction
  - Lidar odometry
  - Lidar mapping
  - Transform integration.

Segmentation

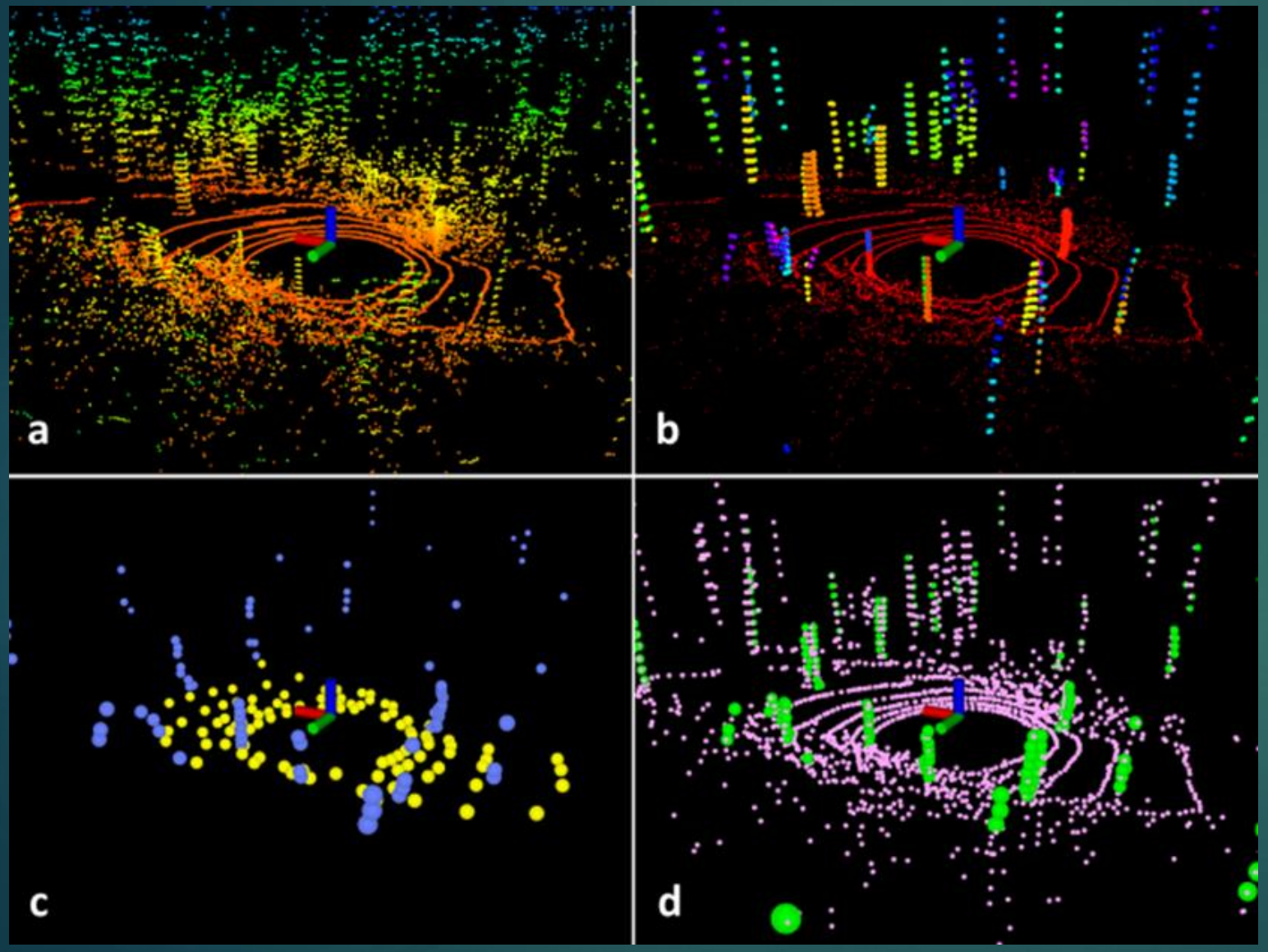
- Let $P_t = \{p_1, p_2, ..., p_n\}$ be the point cloud acquired at time t, where p_i is a point in P_t.
- P_t is first projected onto a **range image**.
  - Resolution 1800x16
- Value r_i represents the Euclidean distance
- Column-wise evaluation provides ground plane estimation, assigns ground points, and keeps those points out of segmentation.
- Each point in the cloud map is given three properties:
  1. ground or segmented point label
  2. a column and row index in the range image
  3. a range value.




```
>> pcread('1649447697.910083055.pcd')
ans =
  pointCloud with properties:

     Location: [0.0093507 3.3871205e-05 0.0614974]
        Count: 1
      XLimits: [0.0093507 0.0093507]
      YLimits: [3.3871205e-05 3.3871205e-05]
      ZLimits: [0.0614974 0.0614974]
        Color: []
       Normal: []
    Intensity: 0
```

Feature Extraction



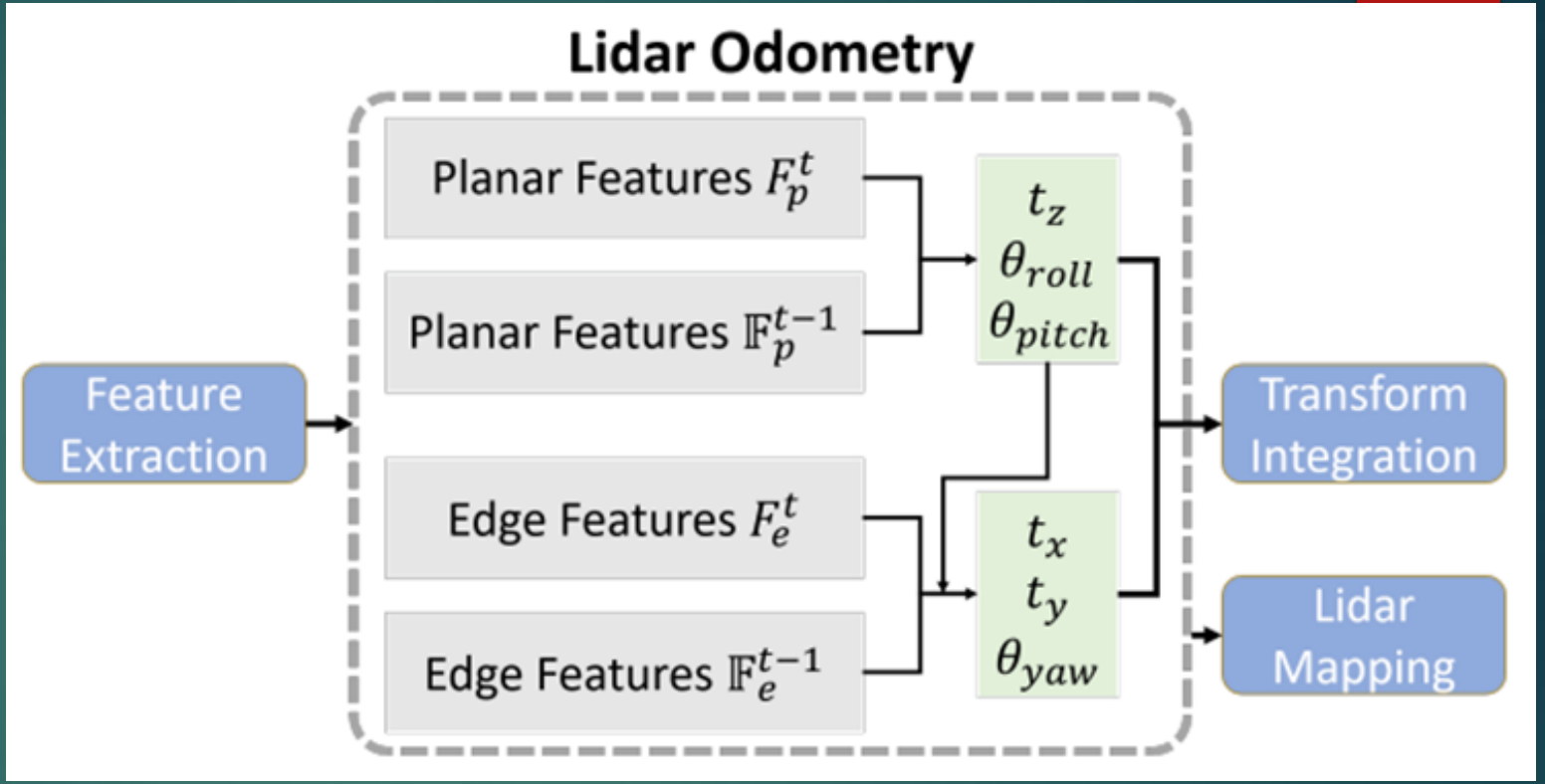
- Let S be the set of continuous points of pi from the same row of the range image.
  - LeGO-LOAM sets |S| to 10.
- Knowing S and r_i we use equation (1) to evaluate our features, the roughness, c.

$$c = \frac{1}{|S| \cdot \|r_i\|} \Big\| \sum_{j \in S, j \neq i} (r_j - r_i) \Big\|. \qquad (1)$$

- Threshold for c:
  - the n_{Fe} edge feature points with maximum c that are not ground points
  - n_{Fo} planar feature points with minimum c that can be either ground or segmented points.
  - The 360 degrees range image is divided into 6 sub-images of resolution 300 by 16. And we define F_e and F_p as the set of all edge and planar features from all sub-images.
- And n_{Fe}, n_{Fo}, n_{FFe}, n_{FFp} are chosen to be 2, 4, 40, 80.

Lidar Odometry

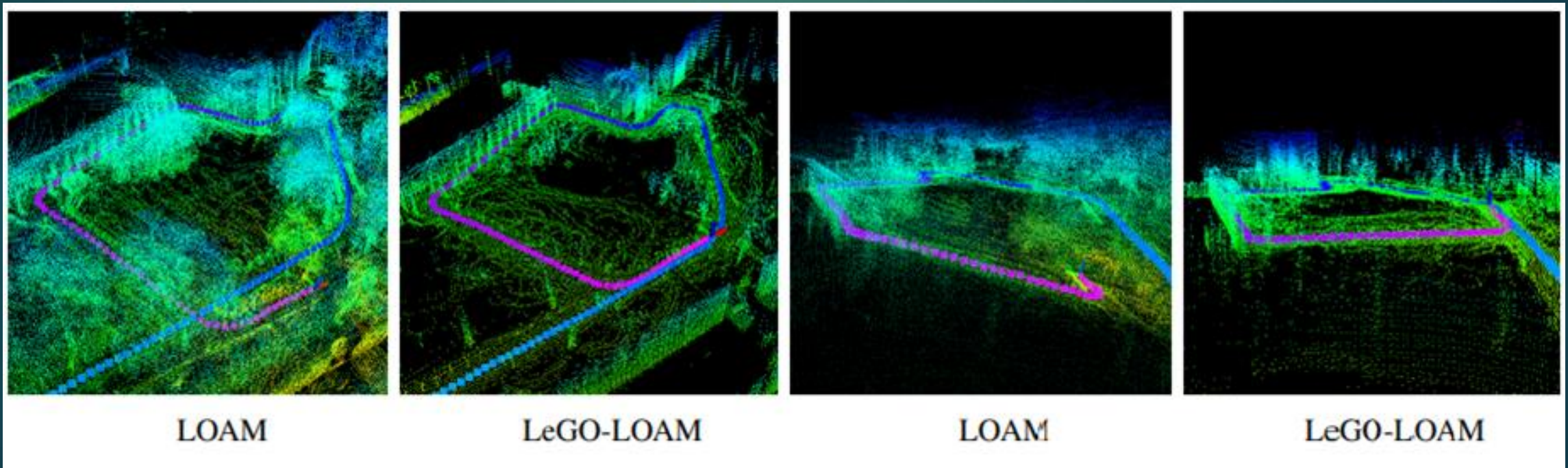
- Lidar Odometry estimates sensor motion between consecutive scans.
- Two methods to improve accuracy & efficiency for transformation T:
  - Label matching
  - Two-step Levenberg-Marquardt optimization
    - Acquire [t_z, θ_{roll}, θ_{pitch}] through estimation of matching planar features
    - Then acquire [t_x, t_y, θ_{yaw}] through estimation of edge features and the previous parameters



- Final 6D transformation of two scans by fusing the six parameters we acquired
- 35% reduction in computational power for similar accuracy to other methods

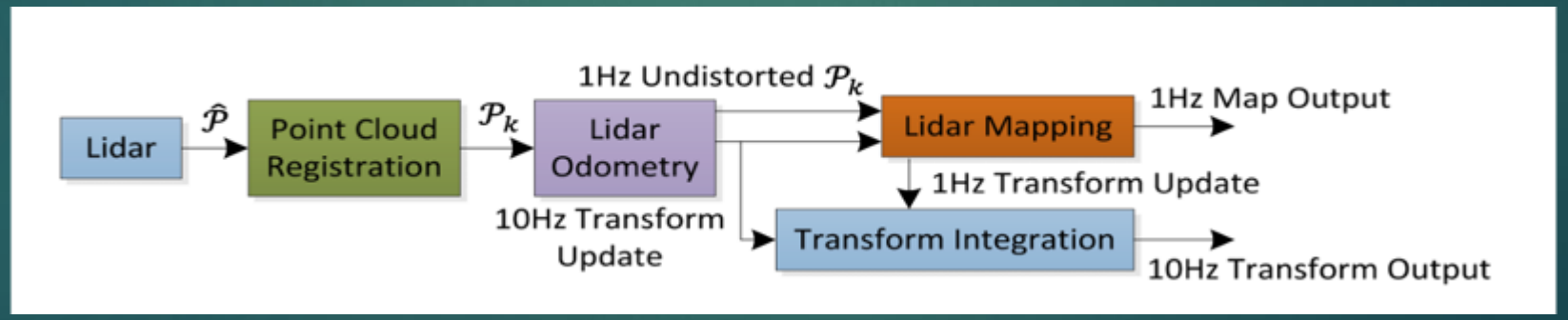
Lidar Mapping

- We further refine the odometry using previously acquired matched features $\{F^t_e, F^t_p\}$ in a surrounding point cloud map Q^{t-1}
  - Run L-M algorithm again at lower frequency
- LeGO-LOAM stores point cloud maps differently from other mapping software which reduces computational time and data storage.

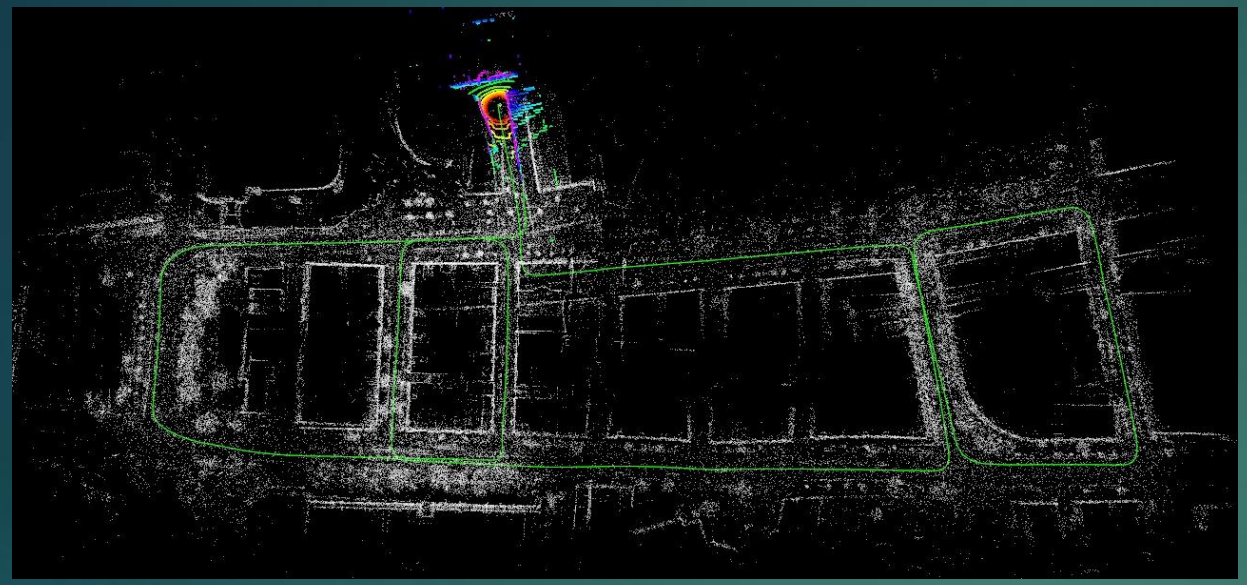


LOAM: Lidar Odometry and Mapping in Real-time

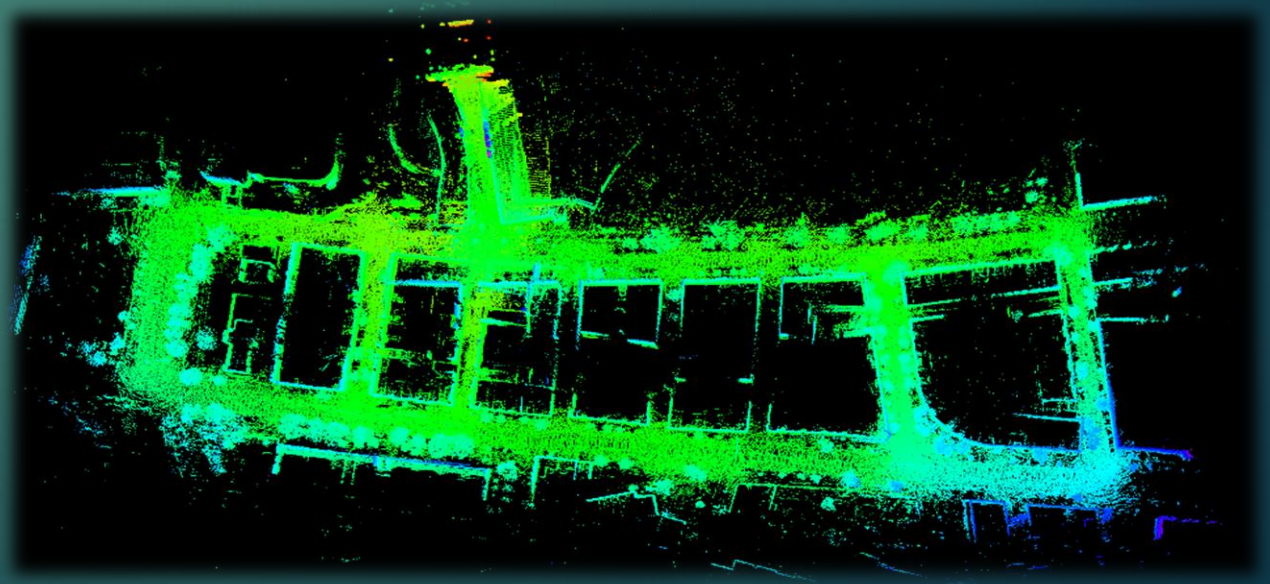
- Loam is a two-step Levenberg-Marquardt optimization method that uses the planar and edge features to solve different components of the six degree-of-freedom transformation across consecutive scans.
- The method achieves both low-drift and low-computational complexity without the need for high accuracy ranging or inertial measurements.
- A-LOAM is an Advanced implementation of LOAM which uses Eigen and Ceres Solver to simplify code structure.



Comparing A-LOAM with LeGO-LOAM

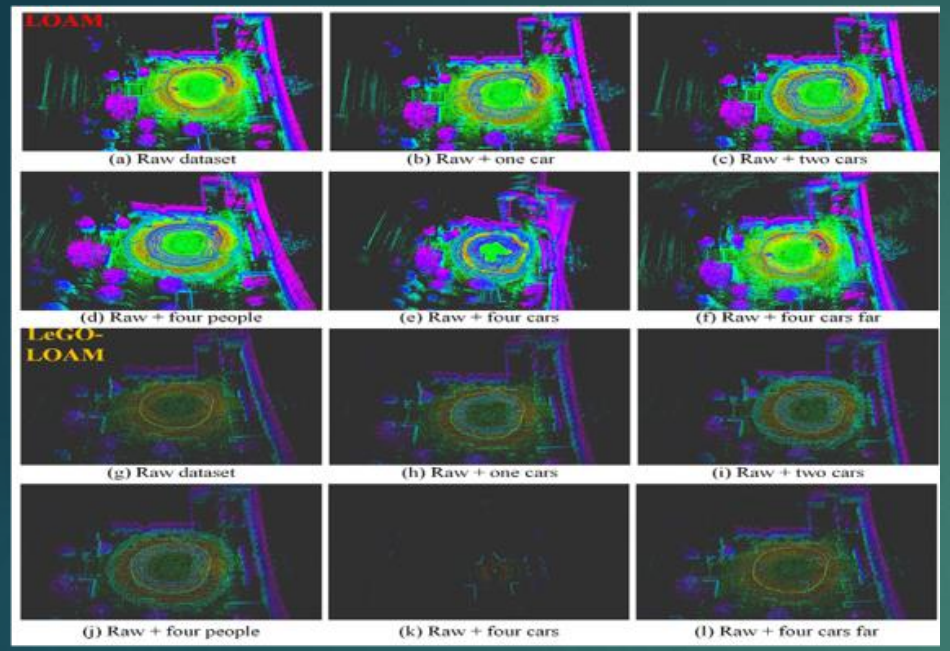


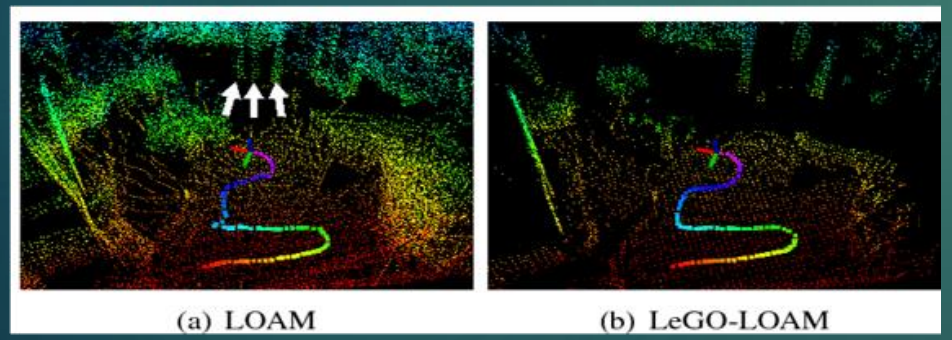
Projections using A-LOAM



Projections using LeGO-LOAM with imu data

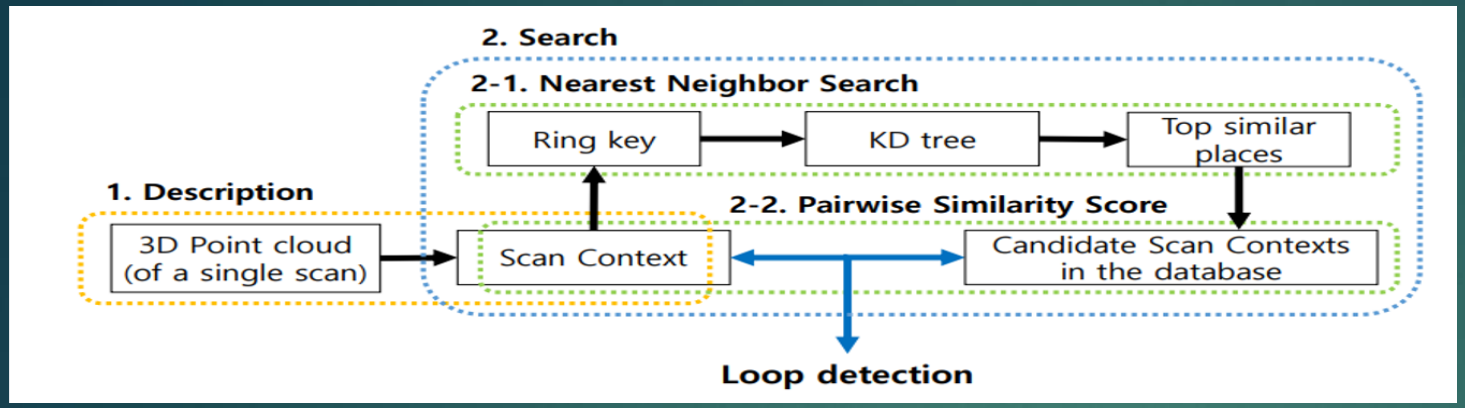
How LeGO-LOAM is better than LOAM



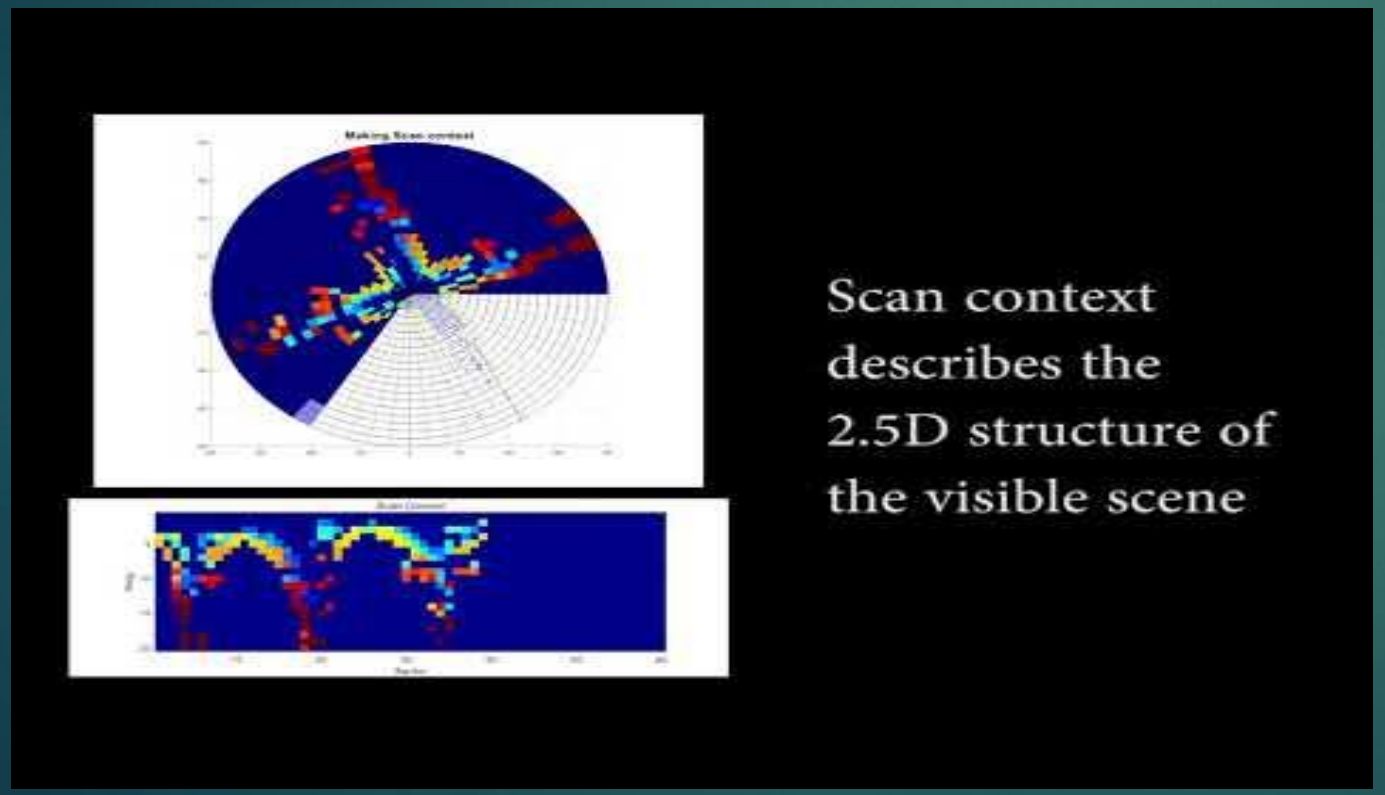


- LeGO-LOAM has tight coupling, compared to LOAM
- Inertial Measurment Unit is used in LeGO-LOAM
- The ground plane constraint in LeGO-LOAM uses scan-to-scan-match odometry rather than scan-to-map mapping, making the odometry faster than LOAM.
- The total number of feature points extracted in LeGO- LOAM are less than LOAM, giving more vivid data.

SC LeGO-LOAM: ScanContext



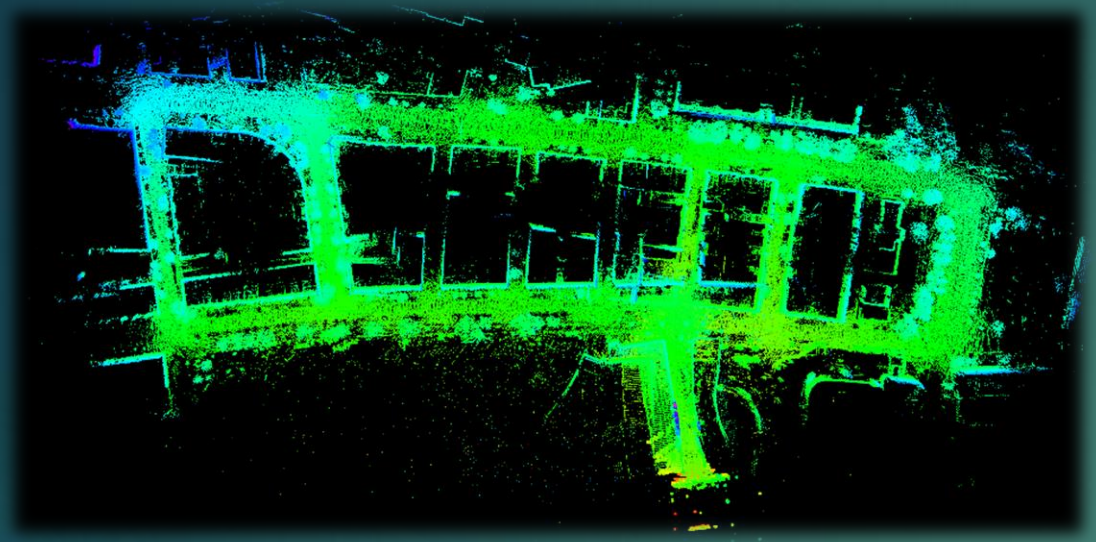
Pipieline of ScanContext

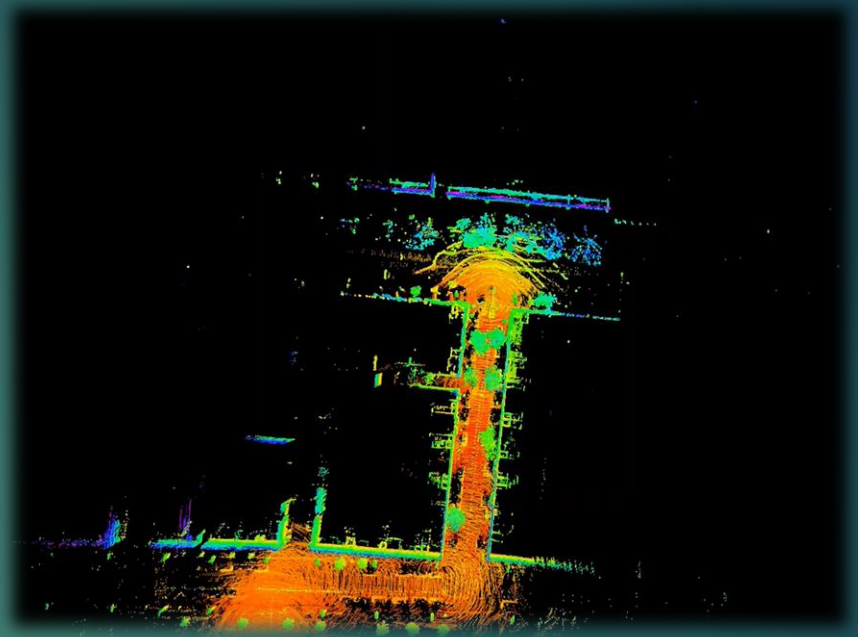


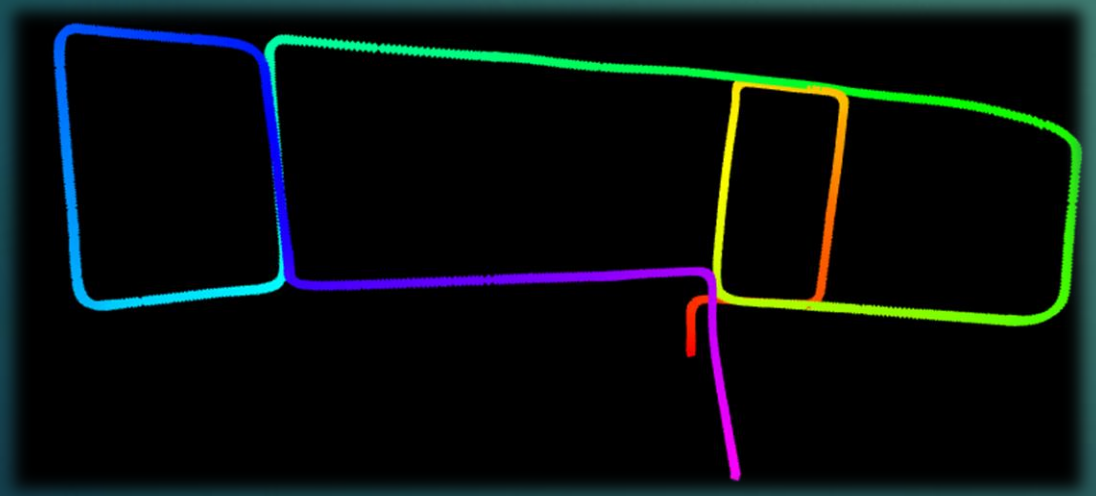
- Scan Context is a global descriptor for LiDAR point cloud, which is especially designed for a sparse and noisy point cloud acquired in outdoor environment.

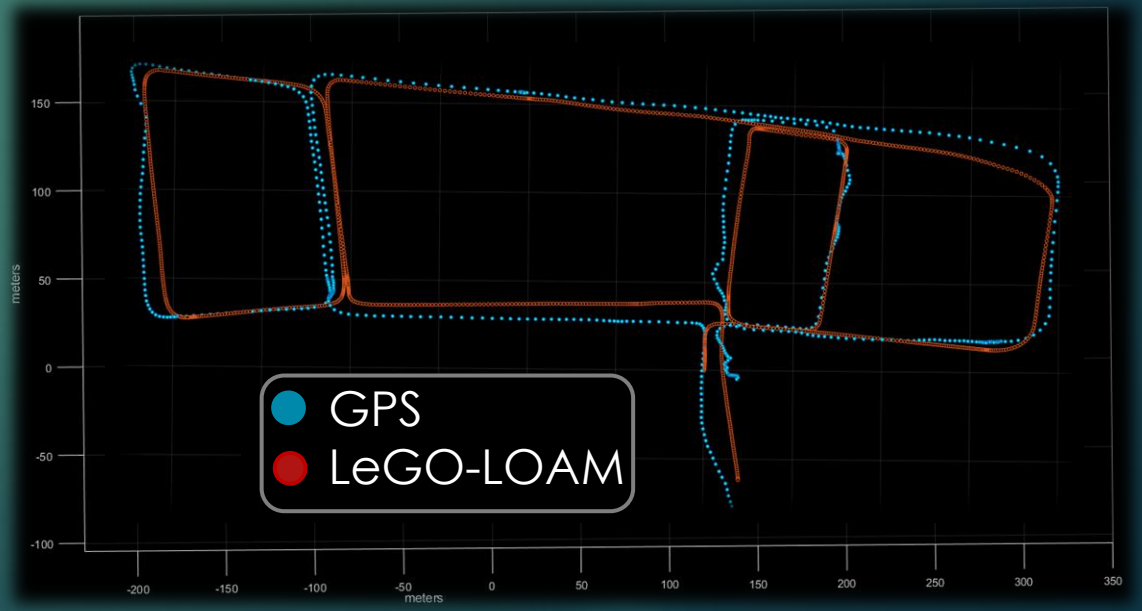
Results

Without Loop Closure





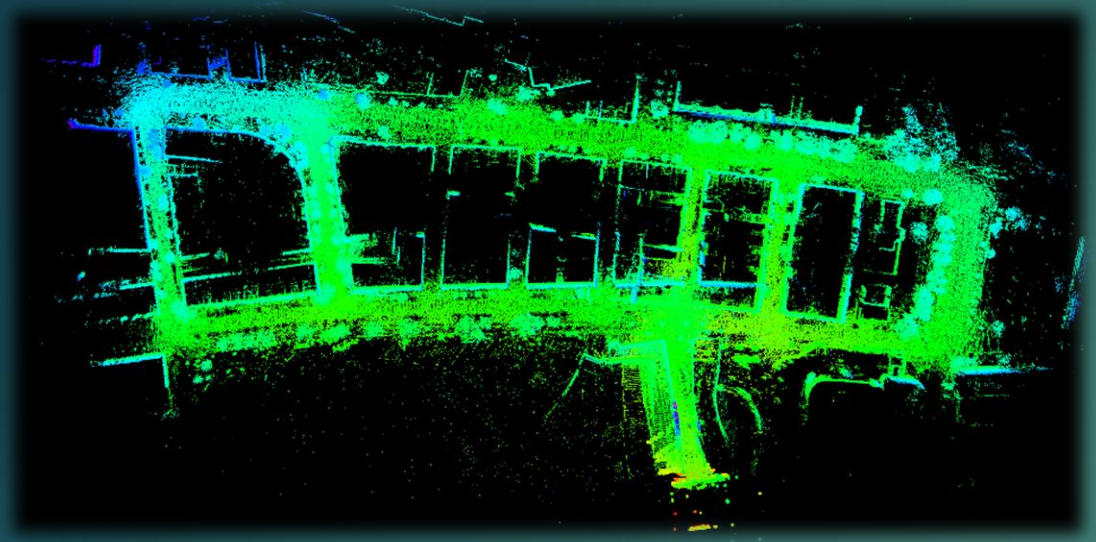




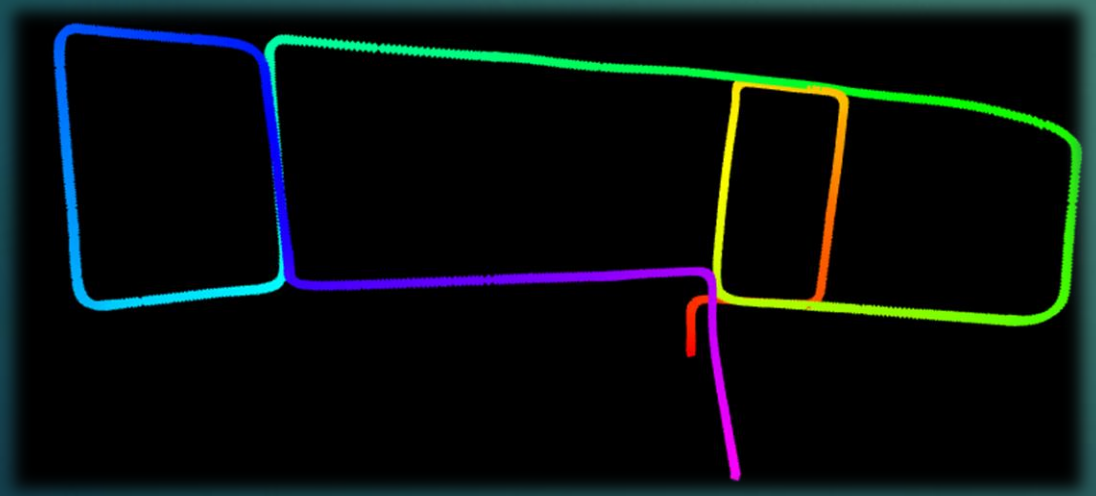
Results

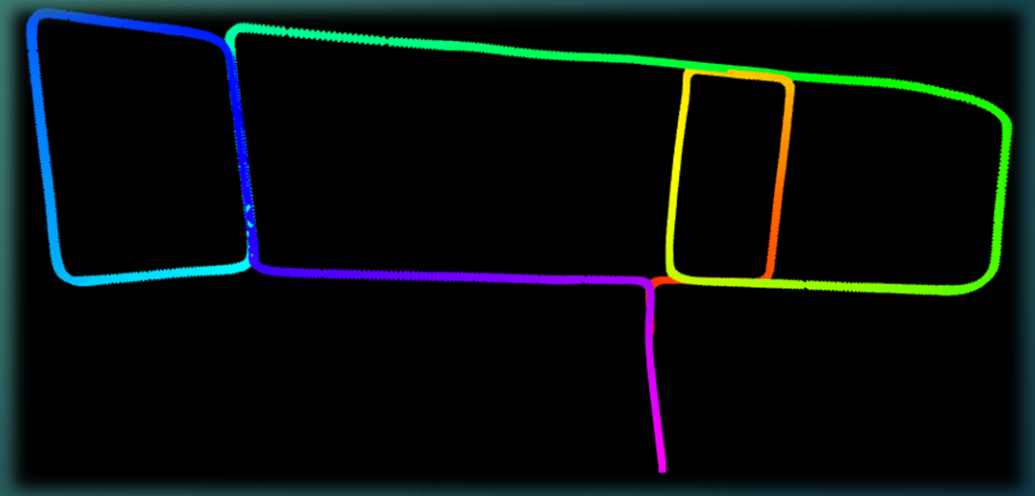
Without Loop Closure

With Loop Closure



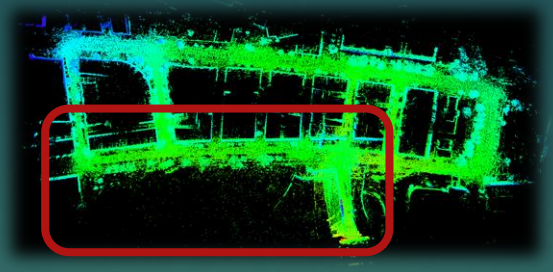




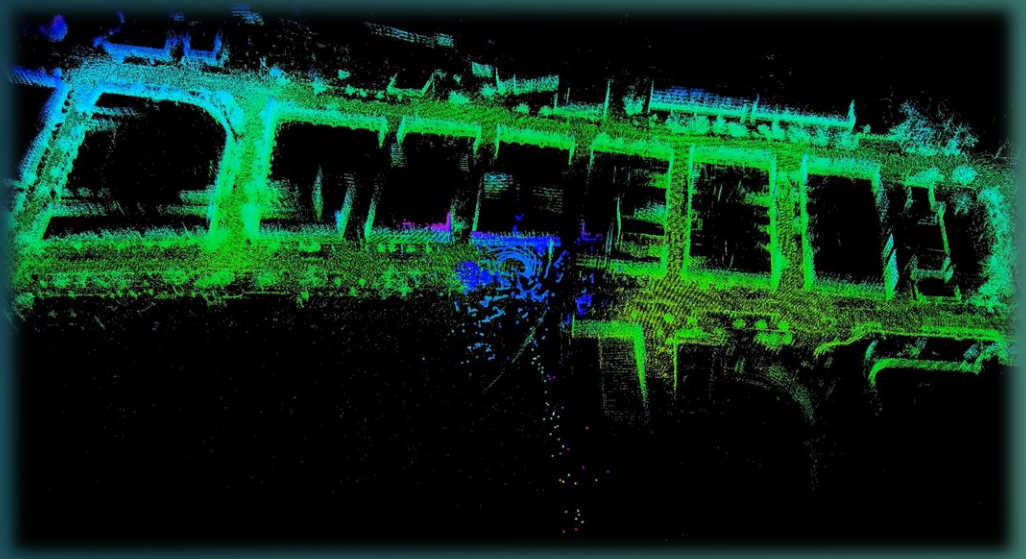


Results

Without Loop Closure



With Loop Closure





Using loop closure, LeGO-LOAM was able to adjust the altitude drift.

Results

Is loop closure good enough?

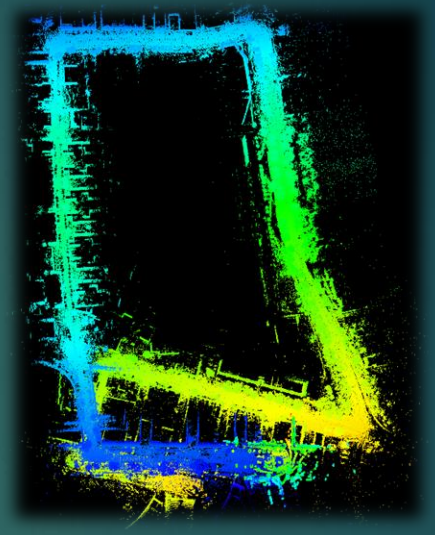
Pushing LeGO-LOAM to its limit

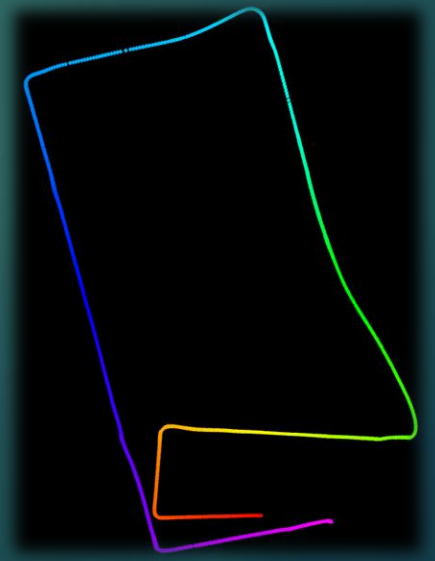


- In the utility.h file, change *globalMapVisualizationSearchRadius from 500m to 1000m*

Even **with** loop closure LeGO-LOAM fails

LeGO-LOAM with loop closure

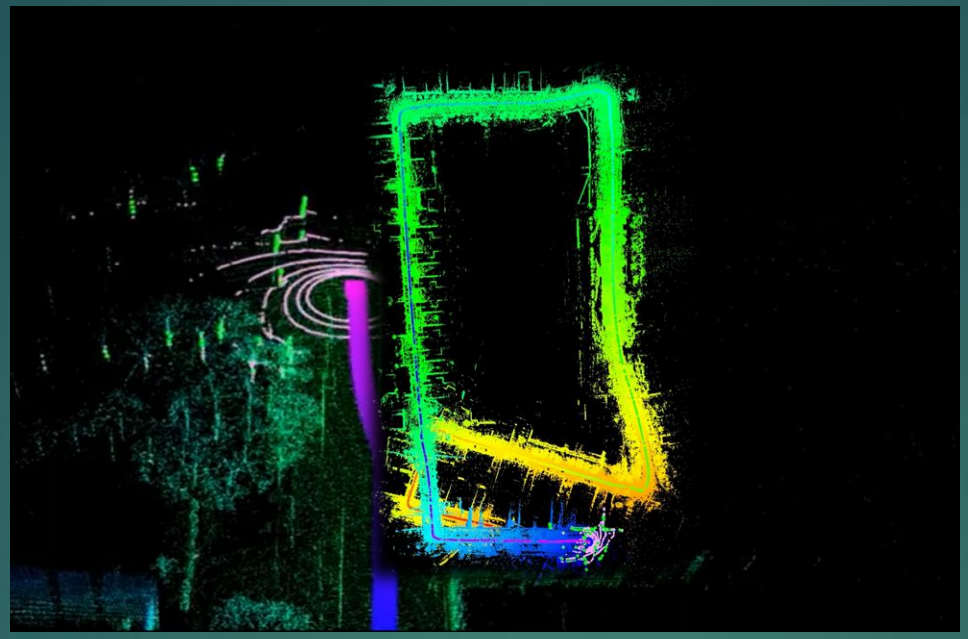




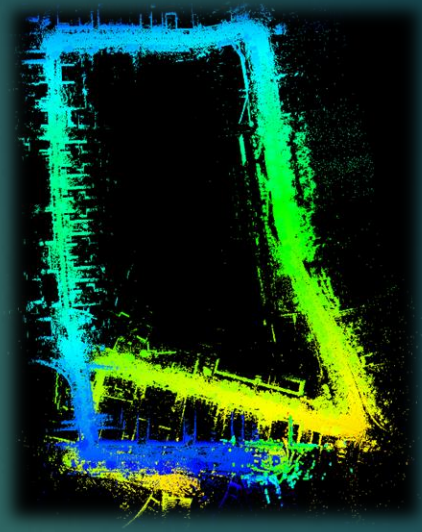
Results

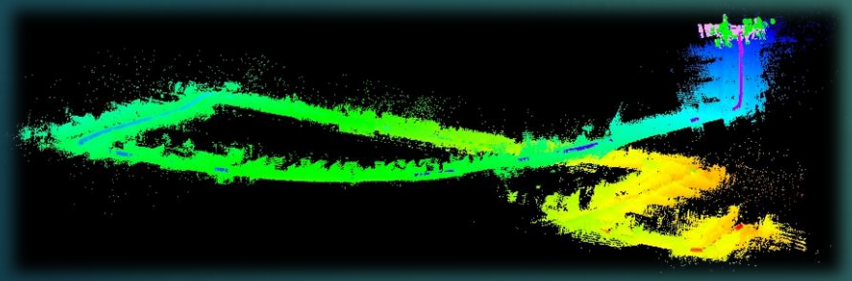


SC LeGO-LOAM

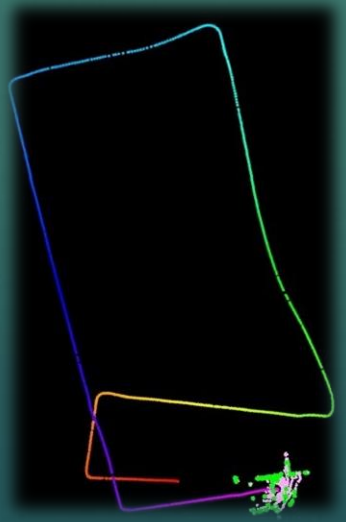


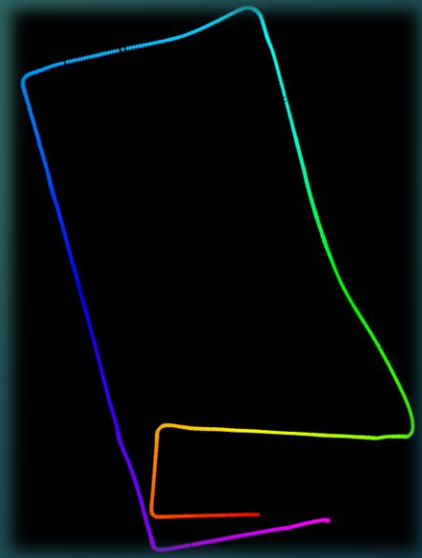
LeGO-LOAM
with loop closure





SC LeGO-LOAM also fails





Conclusion

- LiDAR SLAM = LiDAR Odometry (LeGO-LOAM) + Loop detection (Scan Context) and closure (GTSAM)
- LeGO-Loam can handle loop closures over small distances and static environment
- LeGO-Loam falis at dynamic environment and long distances
- Loop closure can adjust errors in altitude estimation as well
- Using Lidar alone and even with imu, LeGO-LOAM and SC LeGO-LOAM both fail to perform loop closure at dynamic environment

Future scope

- Dynamic obstacle feature removal
  - Increased Lidar mapping accuracy
  - Lidar odometry localization improvement

Questions?

